

Interview prep set 2

1. What is the difference between list, tuple, set, and dictionary in Python?

LIST	TUPLE	DICTIONARY	SET
Allows duplicate members	Allows duplicate members	No duplicate members	No duplicate members
Changeable	Not changeable	Changeable, indexed	Cannot be changed, but can be added, non-indexed
Ordered	Ordered	Unordered	Unordered
Square bracket []	Round brackets ()	Curly brackets{ }	Curly brackets{ }

Note – Explain each by an example in interview

Example -**Mutable vs Immutable Objects**

- **Mutable** → Can change in place
- **Immutable** → Cannot change after creation

```
# Mutable
lst = [1,2]
lst.append(3)

# Immutable
x = 10
x = x + 1 # creates new object
```

2. How does Python manage memory? What is **garbage collection**?

Python manages memory automatically using a private heap, reference counting, and a generational garbage collector to reclaim unused objects and handle circular references efficiently.

Python manages memory **automatically** using a combination of:

1. **Private Heap Space**

- All Python objects (variables, lists, dicts, functions, classes) are stored in a **private heap**
- The heap is **managed internally** by Python
- Developers **cannot directly access** this memory

```
a = [1, 2, 3]
b = {"id": 10}
```

Both a and b live in Python's private heap.

2. Memory Manager

- Allocates memory when objects are created
- Deallocates memory when objects are no longer
- Uses **object-specific allocators** for:
 - Integers
 - Strings
 - Lists
 - Dictionaries
- Small objects are optimized using **pymalloc** (Python's internal allocator).

3. Reference Counting (Primary Mechanism)

- Every object has a **reference count** — how many variables point to it.

```
a = [1, 2, 3]    # ref count = 1
b = a            # ref count = 2
del a            # ref count = 1
del b            # ref count = 0 → memory eligible for clean
```

- Cannot handle circular references

4. Garbage Collection (GC)

Why GC is Needed?

Reference counting **fails for circular references**:

```
a = {}
b = {}
a['b'] = b
b['a'] = a
```

- a references b
- b references a
- Reference count never reaches 0
- **Garbage Collector** handles this.

How Garbage Collection Works

Python uses **Generational Garbage Collection**:

Generations:

Generation	Description
Gen 0	Newly created objects
Gen 1	Objects surviving Gen 0
Gen 2	Long-lived objects

Process:

1. New objects → Gen 0
2. If they survive cleanup → move to Gen 1
3. Long-living objects → Gen 2
4. GC runs **less frequently** on older generations

This improves performance

You **do not manually allocate or free memory** like in C/C++.

3. Explain **mutable vs immutable** objects with examples.
Refer question 1

4. What is the difference between == and is?

- == → value comparison
- is → memory reference

```
a = [1,2]
b = [1,2]

a == b    # True
a is b    # False
```

5. How do you reverse a list **without using reverse()**?

```
lst = [1,2,3]
rev = lst[::-1]
```

6. What is **list comprehension**? Rewrite a loop using list comprehension.

```
squares = [x*x for x in range(5)]
```

```
squares = []
for x in range(5):
    squares.append(x*x)
```

7. Explain **shallow copy vs deep copy**.
- a. Shallow → copies reference
 - b. Deep → copies entire object

```
import copy

a = [[1,2]]
b = copy.copy(a)
c = copy.deepcopy(a)
```

8. How do you handle **exceptions** in Python?

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Done")
```

9. What is the use of ***args** and ****kwargs**?

`*args` and `**kwargs` allow a function to accept a variable number of arguments.

Term	Meaning
<code>*args</code>	Variable positional arguments
<code>**kwargs</code>	Variable keyword arguments

They make functions **flexible and reusable**, which is very common in **frameworks, libraries, and data pipelines**.

10. Difference between **append()** vs **extend()** in lists.

Append() → Adds the **entire object as a single element** at the end of the list

```
lst = [1, 2, 3]
lst.append([4, 5])

print(lst)
```

```
[1, 2, 3, [4, 5]]
```

← output

Extend() → Adds **each element** of the iterable individually to the list

```
lst = [1, 2, 3]
lst.extend([4, 5])

print(lst)
```

```
[1, 2, 3, 4, 5]
```

← output

11. What is a **lambda function**? Give a real use case.

A **lambda function** is a **small, anonymous function** defined using the lambda keyword.

- It can have **any number of arguments**
- But **only one expression**
- It returns the result **automatically**

Syntax: lambda arguments: expression

```
square = lambda x: x * x
print(square(5))    # 25
```

12. How do you remove duplicates from a list efficiently?

The most efficient way is using a set. If order must be preserved, we use a set to track seen elements while iterating the list.

The **best approach depends on requirements**:

- Do you need to **preserve order**?
- Is the list **large**?
- Are elements **hashable**?
- Most Efficient (If Order Does NOT Matter)

Using `set()` – Fastest

Time: **$O(n)$**

Space: **$O(n)$**

Order is **NOT** preserved

- Efficient AND Order Preserved Using `set()` + loop

```
def remove_duplicates(nums):
    seen = set()
    result = []
    for n in nums:
        if n not in seen:
            seen.add(n)
            result.append(n)
    return result
```

```
remove_duplicates([1,2,3,2,4,1])
# [1, 2, 3, 4]
```

Time: **$O(n)$**

Space: **$O(n)$**

Order preserved

13. What are **Python decorators** (high-level explanation)?

A **Python decorator** is a way to **add extra behavior to a function or method without changing its original code**.

In Lemon words we can say:

A decorator **wraps** another function to extend its behavior.

Decorators help when you want to add **common functionality** like:

- Logging
- Authentication
- Authorization
- Timing
- Caching
- Retry logic

...without repeating code everywhere.

14. What is the difference between `range()` and `xrange()`? (*Python 2 vs 3 awareness*)

- Python 2 Behavior

Function	Behavior
<code>range()</code>	Creates a list in memory
<code>xrange()</code>	Creates a generator-like object

Key Difference:

- `range()` → **memory heavy**
- `xrange()` → **memory efficient**

- Python 3 Behavior

- `xrange()` **was removed**
- `range()` now behaves like **Python 2's `xrange()`**
It does **not** store all values in memory.

15. How does Python handle **default function arguments**?

Default arguments are evaluated **once when the function is defined**, not each time it is called.

```
def greet(name="Guest"):
    print(f"Hello {name}")

greet()          # Hello Guest
greet("Anand")   # Hello Anand
```

Python evaluates default function arguments at definition time. Mutable defaults can cause shared state issues, so `None` is commonly used as a safe default.